

TECHNICAL SPECIFICATION

CURRENT FREE CASH FLOW YIELD

Status: Implemented (V1.0.0) **Companions:** `docs/current-fcf-yield-prd.md`, `docs/current-fcf-yield-design.md` **Audience:** Implementing engineers / agents

1. Scope

This spec defines the concrete contracts for the current free cash flow yield feature: the new persistence table, the repository trait, the read-time payload and assembly function, the two IPC commands, the TypeScript bindings, and the file-level change table. It documents the implementation **as shipped**.

2. Data model

2.1 New table — `current_price`

Added to `src-tauri/src/db/migrations/V1__initial.sql` as an additive `CREATE TABLE IF NOT EXISTS`. The schema file is applied idempotently via `execute_batch` on every open, so existing databases pick up the new table automatically with no separate migration step.

```
CREATE TABLE IF NOT EXISTS current_price (  
  cik          TEXT PRIMARY KEY REFERENCES company(cik) ON DELETE RESTRICT,  
  ticker       TEXT NOT NULL,  
  price_micro  INTEGER NOT NULL,    -- spot price in USD micro-units (price x 1e6)  
  as_of        TEXT NOT NULL,      -- RFC3339 timestamp of the fetch  
  source       TEXT NOT NULL       -- provider tag, e.g. "yahoo"  
);
```

One row per company (the `cik` primary key enforces it); a new fetch performs an upsert. The `opens_and_runs_migrations` test's expected-tables list includes `current_price`.

2.2 Units

- `price_micro`, `market_cap_micro`, `ttm_fcf_micro` — USD × 1e6 (i64).

- `fcf_yield_micro` — decimal ratio $\times 1e6$ (e.g. `118_000` = 11.8%); may be negative.
 - `shares` — a **raw** basic share count (not micro-units).
-

3. Rust contracts

3.1 Repository — `src-tauri/src/repos/current_price.rs`

```
pub struct CurrentPrice {
    pub price_micro: i64,
    pub as_of: chrono::DateTime<chrono::Utc>,
    pub ticker: String,
}

#[async_trait]
pub trait CurrentPriceRepo: Send + Sync {
    async fn upsert(
        &self, cik: &Cik, ticker: &str, price_micro: i64,
        as_of: DateTime<Utc>, source: &str,
    ) -> Result<(), RepoError>;

    async fn get(&self, cik: &Cik) -> Result<Option<CurrentPrice>, RepoError>;
}

pub struct SqliteCurrentPriceRepo { /* pool: Arc<Pool> */ }
```

`upsert` uses `INSERT ... ON CONFLICT(cik) DO UPDATE SET ...`. The struct mirrors `SqliteHistoricalPriceRepo` (same pool guards, params, error type). Registered via `pub mod current_price;` in `repos/mod.rs`. Unit tests cover upsert-then-get and upsert idempotency (overwrite).

3.2 Metric enum — `src-tauri/src/domain/metric.rs`

Added `Metric::CurrentFreeCashFlowYield` $\leftrightarrow$ `"current_free_cash_flow_yield"` in the enum, `as_str`, `from_str`, and `ALL`. It is **not** instant. (`CurrentMarketCap` already existed.) The `concept_map.rs` `match over Metric` gains a non-fact arm for the new variant.

3.3 Read payload & assembly — `src-tauri/src/derived/series.rs`

`ReadCtx` gains a borrowed repo reference:

```
pub struct ReadCtx<'a> {
    pub normalized_facts: &'a dyn NormalizedFactRepo,
    pub derived_metrics: &'a dyn DerivedMetricRepo,
    pub prices: &'a dyn HistoricalPriceRepo,
    pub current_prices: &'a dyn CurrentPriceRepo, // new
}
```

Payload and entry point:

```
#[derive(Debug, Clone, Serialize)]
pub struct CurrentValuation {
    pub price_micro: i64,
    pub price_as_of: chrono::DateTime<chrono::Utc>,
    pub shares: i64,
    pub shares_period_end: chrono::NaiveDate,
    pub market_cap_micro: i64,
    pub ttm_fcf_micro: i64,
    pub ttm_fcf_period_end: chrono::NaiveDate,
    pub fcf_yield_micro: i64,
}

pub async fn current_valuation(
    ctx: &ReadCtx<'_>, cik: &Cik,
) -> Result<Option<CurrentValuation>, AppError>;
```

Algorithm (returns `Ok(None)` at any missing input):

1. `ctx.current_prices.get(cik)` → stored spot price, else `None`.
2. Latest basic shares via `current_series(ctx, cik, SharesOutstandingBasic).last()`.
3. `market_cap = super::market_cap(price_micro, shares); None` if `<= 0`.
4. Numerator: latest quarterly `free_cash_flow_ttm_series` point; fallback to latest annual `free_cash_flow` point; else `None`.
5. `super::fcf_yield_micro(numerator, market_cap); None` if undefined.

Serde uses default **snake_case** field names — the wire contract the TypeScript bindings mirror exactly.

The `revenue_aware_series` dispatcher gains an explicit early arm:

```
Metric::CurrentMarketCap | Metric::CurrentFreeCashFlowYield => return Ok(Vec::new()),
```

documenting that these are live scalars served by `current_valuation`, not series.

3.4 Orchestrator — `src-tauri/src/pipeline/orchestrator.rs`

`IngestionDeps` gains `pub current_prices: Arc<dyn CurrentPriceRepo>`. In `fetch_prices`, after the historical-close upsert loop, the spot price is fetched best-effort:

```
match deps.market_data.current_price(ticker).await {
    Ok(p) => deps.current_prices
        .upsert(cik, &ticker.0, p, chrono::Utc::now(), "yahoo").await?,
    Err(e) => /* record Warn event, continue */,
}
```

A spot-price failure never fails ingestion.

3.5 IPC — `src-tauri/src/ipc/commands.rs`, `state.rs`, `mod.rs`

```
#[tauri::command]
pub async fn get_current_valuation(
    state: State<'_, AppState>, cik: String,
) -> Result<Option<CurrentValuation>, AppError>; // read-only, no network

#[tauri::command]
pub async fn refresh_price(
    state: State<'_, AppState>, cik: String,
) -> Result<Option<CurrentValuation>, AppError>; // live callout, persists, recomputes
```

`refresh_price` resolves the company's ticker, calls `state.market_data.current_price(&ticker)`, upserts into `current_prices`, then returns `current_valuation`. `AppState` gains `current_prices: Arc<SqliteCurrentPriceRepo>`, threaded into both `read_ctx()` and `pipeline_deps()`. Both commands are registered in the `generate_handler!` list in `ipc/mod.rs`.

4. TypeScript contracts — `src/`

4.1 Types — `api/types.ts`

```
export type Metric = /* ... */ | "current_free_cash_flow_yield";

export interface CurrentValuation {
    price_micro: number;
    price_as_of: string; // RFC3339
```

```

shares: number;           // raw count
shares_period_end: string; // ISO date
market_cap_micro: number;
ttm_fcf_micro: number;
ttm_fcf_period_end: string; // ISO date
fcf_yield_micro: number;    // ratio x 1e6, may be negative
}

```

`current_free_cash_flow_yield` is added to `RATIO_METRICS` and given a `METRIC_LABELS` entry (`"Current Free Cash Flow Yield"`).

4.2 Client — `api/client.ts`

```

export async function getCurrentValuation(cik: string): Promise<CurrentValuation | null>;
export async function refreshPrice(cik: string): Promise<CurrentValuation | null>;

```

4.3 Queries — `state/queries.ts`

- `QK.currentValuation(cik)` .
- `useCurrentValuation(cik)` — read query, `enabled: !!cik` .
- `useRefreshPrice()` — mutation; on success sets the `currentValuation` cache directly and invalidates the annual `free_cash_flow_yield` metric history (drives the delta chip).

4.4 UI — `features/dashboard/CurrentValuationCard.tsx`

Props `{ cik }`. Consumes `useCurrentValuation`, `useRefreshPrice`, and `useMetricHistory(cik, "free_cash_flow_yield", "annual")` for the delta. Renders the live hero card and the full loading / null / error / success state matrix (PRD §6.3). Mounted at the top of `CompanyDashboardPage` above the widget grid.

5. File-level change table

File	Change
<code>src-tauri/src/db/migrations/V1__initial.sql</code>	Add <code>current_price</code> table
<code>src-tauri/src/db/mod.rs</code>	Add <code>current_price</code> to expected-tables test
<code>src-tauri/src/repos/current_price.rs</code>	New — <code>CurrentPriceRepo</code> + <code>SqliteCurrentPriceRepo</code> + tests
<code>src-tauri/src/repos/mod.rs</code>	<code>pub mod current_price;</code>

File	Change
src-tauri/src/domain/metric.rs	Add <code>CurrentFreeCashFlowYield</code> variant
src-tauri/src/normalize/concept_map.rs	Non-fact match arm for new variant
src-tauri/src/derived/series.rs	<code>ReadCtx.current_prices</code> , <code>CurrentValuation</code> , <code>current_valuation()</code> , scalar arm
src-tauri/src/pipeline/orchestrator.rs	<code>IngestionDeps.current_prices</code> , best-effort spot fetch
src-tauri/src/ipc/commands.rs	<code>get_current_valuation</code> , <code>refresh_price</code>
src-tauri/src/ipc/state.rs	<code>current_prices</code> field; wire into <code>read_ctx</code> / <code>pipeline_deps</code>
src-tauri/src/ipc/mod.rs	Register both commands
src-tauri/tests/* (6 files)	Construct <code>current_prices</code> in <code>IngestionDeps</code> / <code>ReadCtx</code>
src/api/types.ts	<code>Metric</code> , <code>RATIO_METRICS</code> , <code>METRIC_LABELS</code> , <code>CurrentValuation</code>
src/api/client.ts	<code>getCurrentValuation</code> , <code>refreshPrice</code>
src/state/queries.ts	<code>useCurrentValuation</code> , <code>useRefreshPrice</code> , query key
src/features/dashboard/CurrentValuationCard.tsx	New — live hero card
src/features/dashboard/CompanyDashboardPage.tsx	Mount the card
src/test-mock-auri.ts	Mock <code>get_current_valuation</code> / <code>refresh_price</code> + fixture

6. Testing

6.1 Rust

- `current_price` repo unit tests: upsert-then-get; upsert idempotency.
- Migration test asserts `current_price` exists.
- Metric round-trip test covers `current_free_cash_flow_yield` (`as_str` / `from_str`).
- `current_valuation` correctness is exercised against the production database fixture (real ingested companies) and via the existing live integration tests, which now construct the `current_prices` repo.
- Full suite: `cargo test` green (non-ignored).

6.2 TypeScript

- `npm run lint` (tsc) clean; `npm run build` succeeds; `npm test` (vitest) green.
- Mock harness serves `get_current_valuation / refresh_price` so the card renders under the E2E mock.

6.3 Manual / offline acceptance

- LULU refreshed → current yield $\approx 11.8\%$ at sub-\$12B market cap, distinct from the 6.7% period-end figure (PRD §8 S1).
- Network disabled → previously-refreshed company still renders from the stored price (S2).
- Company with no stored price → "No live quote yet" prompt, no number (S3).